

LSTP

Lattice Semantic Transport Protocol

A specification-first protocol for inspectable semantic communication between humans and AI systems

Protocol: LSTP v0.1

Document status: Revised academic/technical whitepaper - draft protocol publication

Whitepaper revision: 2.0

Project attribution: LSTP project; Phase-1 protocol specification attributed in-repository to Garrick Montgomery

Repository analysis date: 21 September 2026

LSTP state: `main` @ `18d947a45f2db220267f70c7ddb07112fc3181d4`

SLAI state: `SLAI-v.2.3` @ `ce4cfe7b1939bc862725e3cb59f488a5b96b1423`

One meaning. Many carriers. Always inspectable.

This is a design objective, not an empirically established guarantee of semantic equivalence.

Abstract

The Lattice Semantic Transport Protocol (LSTP) is a specification-first proposal for representing and transporting structured semantic content between humans, AI systems, and AI agents. The project addresses a recurring engineering problem in AI-mediated communication: natural-language messages often entangle communicative intent, entities, relations, context, uncertainty, evidence, authority, and output requirements in forms that are convenient for humans but difficult to validate or audit deterministically. LSTP proposes an explicit canonical semantic representation - the **Octad Packet** - together with a compact text carrier called **Lattice**, a JSON Schema, an operator table, and permission semantics.

This whitepaper revises earlier project documentation against the repository state rather than treating prior prose as authoritative. At the analyzed LSTP commit, the formal Phase-1 specification is substantive, but the Python parser, compiler, validator, serializer, runtime, command-line interface, carrier modules, and examples are scaffolds rather than a working reference implementation. No executable conformance test suite or benchmark corpus is present. Accordingly, claims about reduced ambiguity, token efficiency, latency, reasoning quality, hallucination reduction, or interoperability are treated as design hypotheses that require future empirical evaluation.

The current canonical representation contains eight fields: pragmatics, atoms, relations, context, confidence, permissions, evidence, and output. Packet identity, versioning, carrier metadata, and audit-envelope information are conceptually outside the Octad. Permission declarations express requested authority; they do not authenticate a sender, authorize an operation, or enforce policy. The effective authorization boundary remains the consuming runtime.

A mandatory integration analysis against current SLAI v2.3 also shows that LSTP is not yet integrated into SLAI. The required target layout places LSTP at `SLAI/model/LSTP/` and a launcher/adaptor at `SLAI/run_lstp.py`, but neither `SLAI/model/` nor `SLAI/run_lstp.py` exists at

the analyzed SLAI commit. The present paper therefore distinguishes current repository evidence, specification-level requirements, integration targets, and future work throughout.

Keywords: semantic transport; semantic representation; AI-agent communication; domain-specific language; structured context; permissions; provenance; inspectability; Lattice; Octad Packet; SLAI; LANTRA.

Reader's guide and status vocabulary

This paper uses four status terms deliberately:

- **Implemented** means executable repository code demonstrates the capability at the analyzed commit.
- **Specified** means a current formal artifact such as the EBNF grammar, JSON Schema, or protocol specification defines the capability, even if executable code does not yet realize it.
- **Proposed / experimental** means the repository documents a design direction whose behavior is not yet a verified implementation contract.
- **Future work** means the capability is a planned or academically motivated extension rather than a present feature.

This distinction is essential because LSTP is presently specification-led. The paper therefore avoids calling empty Python modules a reference implementation and avoids presenting proposed performance benefits as measured outcomes.

Table of Contents

1. Introduction
2. Problem Definition
3. Design Requirements and Principles
4. Conceptual Development
5. LSTP Architecture
6. Canonical Semantic Representation
7. Lattice Language and Syntax

8. Semantic Transport and Carriers
 9. Context and State
 10. Permissions, Safety, and Auditability
 11. Integration with SLAI v2.3
 12. Implementation State
 13. Evaluation and Validation
 14. Comparison and Related Work
 15. Limitations
 16. Future Work and Roadmap
 17. Discussion
 18. Conclusion
 - References
-

1. Introduction

1.1 Background

Modern AI systems increasingly operate through structured interfaces even when their users communicate through natural language. Tool calls, typed APIs, schemas, state machines, agent messages, and policy objects all exist because free-form text alone does not provide deterministic structure for every downstream operation. At the same time, purely machine-oriented representations can obscure communicative intent and become difficult for human operators to inspect. LSTP is motivated by the gap between these two extremes.

The project proposes a semantic transport layer: a representation in which significant communication dimensions are made explicit before or during transport. The goal is not to prove that a single formal language can remove all ambiguity. Rather, LSTP asks whether semantic structure can be normalized enough to improve inspection, validation, replay, and system integration while retaining multiple carrier representations.

The broader idea resembles established separation patterns in protocol and knowledge-representation design. JSON separates structured data from application semantics (Bray, 2017); JSON Schema supplies a declarative vocabulary for describing valid JSON structures (Wright et al., 2022); RDF separates an abstract graph model from concrete serializations (Cyganiak et al., 2014); and agent communication languages distinguish message-level communicative acts from content languages and application behavior. LSTP does not replace these technologies. It applies a similar separation principle to a project-specific semantic packet intended for AI-mediated communication.

1.2 Scope

The current LSTP v0.1 scope comprises:

- an eight-part canonical semantic representation, the Octad Packet;

- a compact Lattice text syntax defined by EBNF;
- canonical JSON structure constrained by JSON Schema Draft 2020-12;
- an operator table that assigns lexical roles to core Lattice symbols;
- context and reference fields;
- evidence/provenance representation;
- requested permission semantics;
- output requirements;
- an architecture in which semantic content is conceptually separable from transport carrier.

The present repository does **not** yet provide a working parser/compiler/serializer/runtime stack or demonstrated SLAI integration. This whitepaper treats those as implementation targets.

1.3 Objectives

The technical objectives of LSTP are to investigate whether a compact protocol can:

1. make communicative intent and semantic structure more inspectable;
2. support a canonical representation that can be serialized into multiple carriers;
3. make context, evidence, confidence, permissions, and output requirements explicit;
4. preserve a clear boundary between permission representation and runtime enforcement;
5. enable future human-to-agent and agent-to-agent integration without binding semantics to one model architecture;
6. support conformance testing and semantic round-trip evaluation once a reference implementation exists.

These are objectives and hypotheses. No current benchmark demonstrates that LSTP achieves them better than alternative representations.

1.4 Non-goals

LSTP v0.1 is not presented as:

- a universal replacement for natural language;
- a universal ontology;
- a cryptographic security protocol;
- an authentication or authorization system;
- a guarantee of safe execution;
- a proof that ambiguity can be eliminated;
- a model-training method;
- a reasoning engine;
- a substitute for LANTRA or another language model;
- a complete network transport protocol comparable to TCP, HTTP, or QUIC.

The word *transport* in LSTP refers primarily to the structured conveyance of semantic content, not to packet routing at the network layer.

2. Problem Definition

2.1 Implicit intent and semantic entanglement

A natural-language request can contain several dimensions simultaneously. Consider a user who asks an agent to summarize a report, use only a particular data source, avoid making changes, explain uncertainty, and produce a concise answer. A language model may infer these conditions, but the conditions are not automatically available as independent machine-checkable fields. They may also be lost when one agent paraphrases the request to another.

LSTP therefore treats communicative force, semantic entities, relations, context, confidence, permissions, evidence, and requested output as distinct representational concerns. The purpose is not to claim that this decomposition is complete, but to make these concerns explicit enough to be inspected and validated.

2.2 Ambiguity

Ambiguity arises at lexical, syntactic, semantic, pragmatic, and contextual levels. Formalization can constrain some ambiguity while creating new requirements: symbols need vocabularies, references need resolvers, and relations need agreed semantics. A formal packet can still encode an ambiguous concept accurately - for example, by recording two candidate interpretations - rather than resolving the ambiguity automatically.

LSTP should therefore be evaluated on **ambiguity management**, not on the claim that ambiguity disappears. A useful protocol should make unresolved alternatives visible and avoid silently inventing specificity that the sender did not provide.

2.3 Context repetition and state drift

Long conversations and multi-agent workflows commonly repeat or paraphrase context. Repetition consumes space and may introduce drift. LSTP includes explicit thread, packet, parent, conversation, turn, binding, temporal, speaker, audience, and reference fields so that

context can be linked rather than copied wholesale. The repository also defines compact context-reference notation such as `↑2` and `↑2@agent_risk` in the operator table.

However, a reference is not useful without a resolver. The current project specifies references but does not implement a context store or resolution algorithm. Resolution therefore remains a host/runtime responsibility.

2.4 Agent coordination

Multi-agent systems need messages that identify intent, content, conversation state, and sometimes authority. Established agent communication research has long separated performatives and conversation metadata from content. LSTP shares this design concern while adopting its own Octad structure. The current project does not provide empirical evidence that LSTP improves coordination relative to FIPA ACL, KQML, JSON messages, typed RPC, or framework-specific agent envelopes.

2.5 Permission ambiguity

A particularly consequential ambiguity is the difference between *requesting* an action and *being authorized* to perform it. The LSTP permissions specification explicitly separates these concerns. A packet may request `EXEC` or `COMMIT`, but the receiver must still evaluate local policy and runtime capability. This is a sound architectural boundary: a serialized field cannot authenticate its own authority.

2.6 Machine interpretability versus human inspectability

Highly compact encodings can improve regularity but become difficult to read. Rich natural language is easy for humans but comparatively difficult to validate structurally. LSTP attempts a middle position: canonical JSON favors explicit structure, while Lattice favors compact text with stable lexical roles. Whether this balance is successful is an empirical usability question that has not yet been tested.

3. Design Requirements and Principles

3.1 Canonical semantic representation

LSTP requires a canonical semantic structure so that carriers can be evaluated against the same logical packet. The v0.1 repository chooses the Octad Packet. A carrier should therefore encode or recover the eight canonical semantic fields without assigning contradictory semantics.

3.2 Inspectability

The project principle “One meaning. Many carriers. Always inspectable.” is best interpreted as a normative target. Inspectability requires:

- named semantic fields;
- explicit confidence rather than only rhetorical hedging;
- explicit evidence objects where provenance matters;
- explicit requested authority;
- explicit context relationships;
- deterministic diagnostics when a packet is invalid;
- preservation of unsupported extensions rather than silent semantic guessing.

Inspectability is not equivalent to correctness. A packet can be perfectly inspectable and still contain false evidence, a poor semantic mapping, or an unauthorized request.

3.3 Recoverability and round-trip integrity

Carrier separation is meaningful only if semantic structure can survive encoding and decoding. This motivates a future invariant:

```
canonical(packet) = canonical(decode(encode(packet)))
```

The equality must be defined carefully. Byte identity may be unnecessary if semantically irrelevant ordering or formatting changes. A reference implementation should therefore specify a canonical comparison model before claiming deterministic serialization or round-trip fidelity.

3.4 Explicit permission semantics

Authority is represented as data and evaluated by the host. Permissions should narrow, never silently broaden, an operation. Forbidden scope should override allowed scope, and unsupported modes should fail closed.

3.5 Structured context

Context should be referential where possible, with explicit identity and lifecycle metadata. This supports replay and audit but requires resolvers and retention policies outside the protocol.

3.6 Extensibility without accidental semantics

The operator table states that undefined draft glyphs must not acquire invented semantics. This is a strong principle for protocol evolution. Extension namespaces, version negotiation, and compatibility rules should eventually make this principle enforceable.

3.7 Carrier independence

LSTP separates semantic content from carrier representation. Carrier independence should not be interpreted as "all carriers are equivalent by definition." Equivalence must be demonstrated by conformance and round-trip tests.

3.8 Evidence before performance claims

A technically credible LSTP project should report measured results only after benchmarks exist. Claims such as lower token use, lower latency, fewer hallucinations, better reasoning, or improved interoperability require comparative experimental designs rather than architectural intuition.

4. Conceptual Development

Earlier LSTP documentation describes a conceptual lineage involving **Lattice**, **HoloSemantics**, and **LΩ**. These terms should be preserved only as design history unless a current normative artifact assigns them executable semantics.

4.1 Lattice

Lattice is the current compact text notation associated with LSTP. Unlike the historical concept alone, it now has concrete Phase-1 artifacts: an EBNF grammar and operator table. This makes Lattice part of the current specification surface, although no repository parser implements the grammar yet.

4.2 HoloSemantics

HoloSemantics is best treated as a historical/conceptual influence: the idea that a compact semantic representation should remain expandable into richer contextual meaning. No current executable module named HoloSemantics establishes a runtime contract. The concept may motivate semantic zoom and structured context, but it should not be described as an implemented subsystem.

4.3 LΩ

LΩ likewise belongs to the conceptual lineage of compact semantic expression and output-oriented structure. In the current canonical model, Ω has a concrete, narrower meaning: the **output** field of the Octad. Historical LΩ concepts should not be conflated with the v0.1 `output` object.

4.4 From concept to protocol

The current repository represents a useful transition from exploratory notation toward protocol engineering. The EBNF grammar, JSON Schema, operator table, and permission specification are materially

stronger foundations than informal examples. The next transition must be from specification to executable conformance, not additional unvalidated syntax expansion.

5. LSTP Architecture

LSTP v0.1: current specification architecture

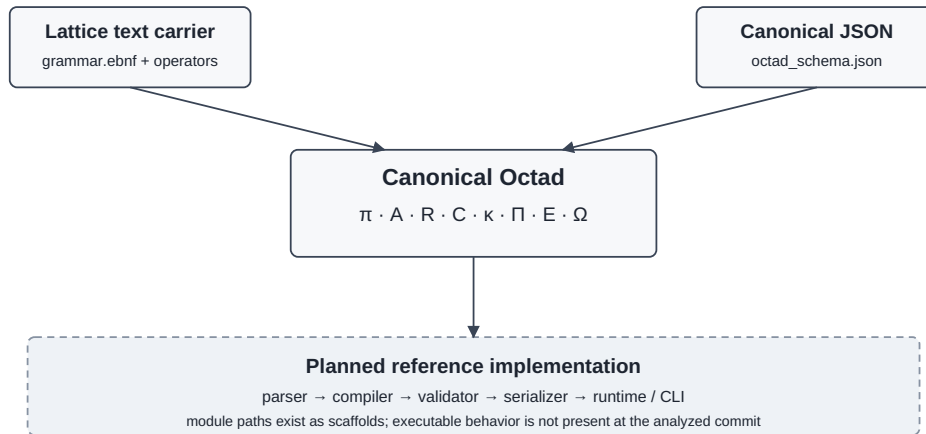


Figure 1. Current LSTP v0.1 specification architecture. Solid elements are substantive specification artifacts. The dashed implementation layer is represented by repository module paths but is not operational at the analyzed commit.

5.1 Semantic layer

The semantic layer is the Octad. It defines the project-specific dimensions that LSTP considers canonical for a message. This is the primary conceptual contract.

5.2 Lattice carrier

Lattice is the compact textual carrier. Its EBNF defines an ordered Octad core as:

```
pragmatics | atoms | relations | context | confidence | permissions | evidence | output
```

Packets may be atomic, framed, named, or streamed according to the grammar. The operator table additionally defines a compact surface notation with directive, interrogative, target, operation, output, confidence, macro, zoom, and context-reference tokens. The

relationship between the compact operator notation and the fully expanded canonical Octad syntax requires implementation-level tests in a future compiler.

5.3 JSON representation

`spec/octad_schema.json` defines the canonical JSON shape using JSON Schema Draft 2020-12. It constrains object properties, enumerations, numeric ranges, identifiers, and required fields. The schema is currently the most machine-checkable artifact in the repository, although the project does not bundle or invoke a validator.

5.4 Validation and diagnostics

The repository names `validator.py`, `diagnostics.py`, `errors.py`, `span.py`, and schema modules, suggesting an intended diagnostic architecture. These files are currently scaffolds. A future validator should preserve source spans, emit stable diagnostic codes, and distinguish lexical, syntactic, schema, semantic, and policy errors.

5.5 Compilation and normalization

A compiler is expected to map Lattice syntax into the canonical Octad representation. Compilation should avoid “helpful” semantic invention. If source syntax is insufficient to determine a relation, permission, or context binding, the compiler should preserve uncertainty or emit a diagnostic.

5.6 Serialization

A serializer should encode a canonical packet into a selected carrier while preserving defined invariants. Deterministic byte serialization has not been specified as a current guarantee; therefore the project should define canonical ordering, numeric formatting, Unicode normalization, and extension behavior before relying on hashes or signatures over serialized bytes.

6. Canonical Semantic Representation

Octad Packet: canonical semantic fields

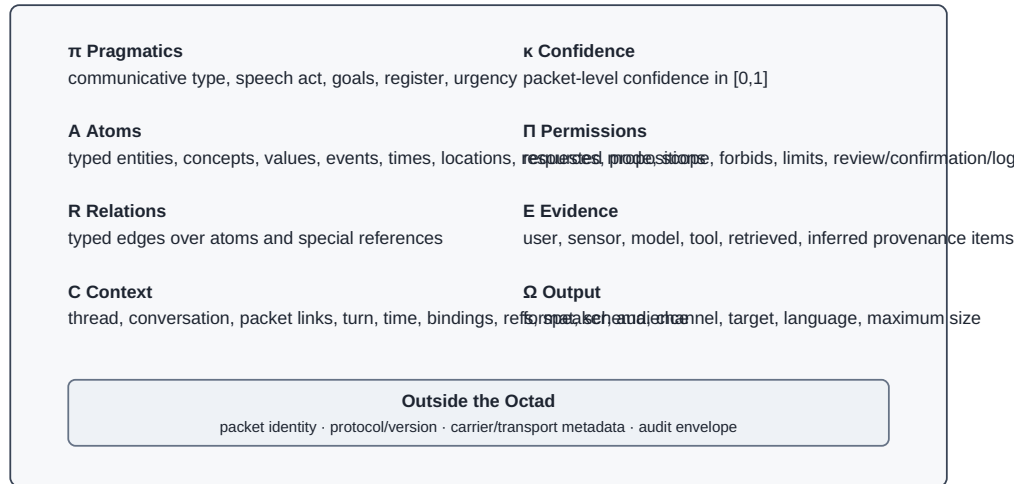


Figure 2. The Octad Packet. Identity, protocol/version, carrier/transport, and audit-envelope information sit outside the eight semantic fields in the v0.1 design.

6.1 Pragmatics (π / pragmatics)

The pragmatics field represents how the message should be understood as a communicative act. The schema requires a `type` and permits speech act, modifiers, goals, register, and urgency. Urgency is bounded from zero to one. Pragmatics should describe communication semantics; it should not be used to smuggle authorization into the packet.

6.2 Atoms (A / atoms)

Atoms are typed semantic units. The schema recognizes entity, concept, value, event, time, location, resource, proposition, and unknown kinds. Atom IDs follow an `aN` pattern such as `a0`. Optional role, datatype, language, attributes, and values provide additional structure.

Atoms are intentionally generic. Domain-specific interpretation still depends on shared vocabulary. This is analogous to the broader ontology problem: portability of syntax does not automatically provide portability of conceptual meaning (Gruber, 1993).

6.3 Relations (R / relations)

Relations connect atoms or special references such as `SELF`, `NOW`, `USER`, and `SYSTEM`. A relation has a type and arguments, and may include relation-level confidence and attributes. The relation vocabulary is not fully standardized by the current repository, so interoperability depends on agreed relation names or extension vocabularies.

6.4 Context (C / context)

Context is anchored by a required `thread_id`. The schema can also represent packet and parent IDs, conversation ID, turn, temporal data, location, bindings, references, speaker, and audience. These fields support replay and continuity but depend on host systems to assign identities and resolve references.

6.5 Confidence (κ / confidence)

The packet-level confidence value is a number from zero to one. The specification does not establish that this value is statistically calibrated. Consumers should therefore treat it as represented confidence unless calibration methodology is supplied by the producer. Future evaluation should distinguish syntactic preservation of confidence from predictive calibration.

6.6 Permissions (Π / permissions)

Permissions represent requested authority. The current modes are `R0`, `RW`, `EXEC`, `PREVIEW`, `COMMIT`, and `SUGGEST`. The permissions object can also express scope, forbids, cost/time limits, confirmation, review, and logging requirements.

The mode is not a capability token. A receiving runtime must intersect the request with local policy and available capabilities.

6.7 Evidence (E / evidence)

Evidence provides provenance-like support for claims or requests. The Lattice grammar includes evidence constructors for user, sensor, model, tool, retrieved, and inferred sources. This is compatible with the broader principle that provenance records should support assessment of how information was produced, while LSTP remains substantially simpler than W3C PROV-DM (Moreau & Missier, 2013).

6.8 Output (Ω / output)

Output describes requested response representation. The grammar supports NL, LATTICE, JSON, YAML, TABLE, CODE, FILE, and NONE, with optional schema, channel, target, language, and maximum byte constraints. A requested format does not guarantee that the receiver can or should satisfy it; inability should be surfaced rather than silently changing semantics.

6.9 Invariants

The JSON Schema requires all eight Octad fields and disallows unspecified top-level properties. Important future implementation invariants should include:

- every relation argument referring to an atom must resolve to an existing atom ID unless it is an allowed special reference;
- context parent links must not create invalid cycles where a host forbids them;
- permissions must not be widened during serialization or translation;
- unknown extension data must not override core permission fields;
- confidence values must remain in the allowed range;
- decode/encode paths must preserve semantic equality under a defined canonicalization function.

Some of these are semantic invariants and are not fully enforceable by JSON Schema alone.

7. Lattice Language and Syntax

7.1 Canonical Octad syntax

The current EBNF defines the canonical text representation. The following example is derived directly from that grammar:

```
[ $\pi$ =(TYPE=REQUEST,SPEECH_ACT=COMMAND)
|A=(a0:ENT("door"){ROLE=TARGET})
|R=(OPEN(a0))
|C=(THREAD="t1")
| $\kappa$ =0.95
| $\Pi$ =(MODE=EXEC,SCOPE=["door"])
|E=(USER("open the door"))
| $\Omega$ =(FORMAT=NL)]
```

This is a **grammar-level example**, not a parser-verified execution trace, because the repository parser is not implemented.

7.2 Framing and naming

The grammar recognizes:

```
atomic_packet = octad_core
framed_packet = "[" octad_core "]"
named_packet  = IDENT ":" framed_packet
stream_packet = named_packet { packet_sep named_packet }
```

This design supports individual packets and named streams while keeping the Octad core ordered.

7.3 Pragmatics syntax

The canonical pragmatics segment uses named entries such as `TYPE`, `SPEECH_ACT`, `MOD`, `GOAL`, `REGISTER`, and `URGENCY`. The operator table also defines compact communicative prefixes such as `!`, `!!`, `?`, and a leading `.`. These compact forms require a compiler mapping into canonical pragmatics before they can be treated as equivalent representations.

7.4 Atoms and relations

Atoms use typed constructors such as:

```
a0:ENT("report"){ROLE=TARGET}
a1:CONCEPT("risk")
```

Relations use function-like relation names:

```
R=(SUMMARIZE(a0), EVALUATE(a1))
```

The syntax does not imply that arbitrary relation identifiers execute host functions. A host must map known semantic operations to capabilities explicitly.

7.5 Context

Canonical context fields include thread, packet, parent, conversation, turn, time, timezone, window, location, bindings, references, speaker, and audience. Compact context references such as `↑2` are defined by the operator table but require a runtime resolver.

7.6 Confidence and approximation

Canonical packet confidence is represented by `κ=number`. The compact operator table additionally assigns `%0.82` as a confidence marker and `~` as an approximation marker. The two are not equivalent: `~` denotes qualitative uncertainty while `%` carries an explicit numeric value.

7.7 Permissions

Canonical permission syntax includes:

```
Π=(MODE=PREVIEW,
  SCOPE=["report"],
  FORBID=["external-write"],
  REQUIRE_CONFIRM=true,
  LOG=true)
```

Permission syntax must be preserved through carrier transformations because accidental widening is a safety-relevant semantic error.

7.8 Output

Canonical output fields specify format and related constraints. The compact operator table defines `->` and semantic zoom markers such as `@z2`. Zoom expresses desired semantic depth, not a guaranteed token count.

7.9 Macros and extensions

Macro reference syntax exists, but macro expansion is deferred. The operator table is explicit that unresolved macros must not be silently replaced with guessed content. This principle should generalize to all unsupported extensions.

7.10 Syntax discipline

Earlier illustrative LSTP prose used overloaded or undefined symbols. The current operator table is intentionally stricter: undefined postfix trend glyphs, for example, should not acquire invented meanings. This improves protocol discipline and should be retained in the implementation.

8. Semantic Transport and Carriers

8.1 Meaning versus carrier

The central architectural claim of LSTP is a separation:

```
semantic packet != carrier representation
```

A semantic packet describes meaning as represented by the Octad. A carrier determines how that structure is encoded for transport or inspection. In the present repository, Lattice and canonical JSON are the meaningful specification targets.

8.2 Current carriers

Lattice text

Lattice is specified by EBNF and operator semantics. It is compact and human-inspectable for technically trained users, but no parser proves conformance yet.

Canonical JSON

Canonical JSON is constrained by JSON Schema. JSON itself is a standardized, language-independent interchange syntax (Bray, 2017). The LSTP schema adds project-specific semantics but should not imply that generic JSON systems understand those semantics automatically.

8.3 Conceptual carriers

Earlier project material discusses broader carrier possibilities, including natural-language or multimodal representations. At the analyzed commit these are **conceptual/future**. A carrier should not be described as supported until it has:

- a formal mapping to the Octad;
- encoder/decoder implementations;
- positive and negative fixtures;

- semantic round-trip tests;
- extension/version behavior;
- documented failure modes.

8.4 Carrier negotiation

The current repository does not implement carrier negotiation. A future protocol envelope should identify protocol version, carrier profile, canonicalization rules, and extension namespaces. Negotiation must remain outside the Octad because these are transport/envelope concerns rather than semantic fields.

9. Context and State

9.1 Packet relationships

`thread_id`, `packet_id`, `parent_packet_id`, and `conversation_id` can express message relationships. This allows a host to construct conversation graphs rather than relying only on chronological text concatenation.

9.2 Context references

The operator table defines stack-depth references such as `↑0`, `↑2`, and `↑2@agent_risk`. These are parseable references whose meaning depends on a host-managed context namespace. Phase-2 parsing may preserve them without resolving them; semantic resolution belongs to later runtime work.

9.3 State ownership

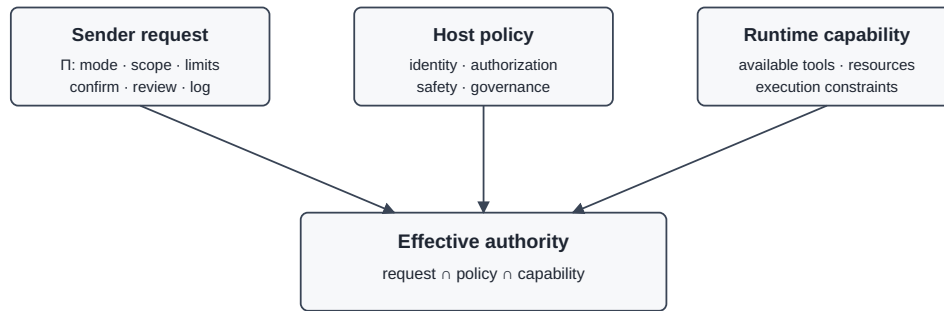
LSTP should not own all conversation state. A protocol packet can reference state, but the authoritative lifecycle, retention, privacy, and mutation rules belong to the consuming environment. This is especially important in SLAI integration, where agent/task context already has its own runtime semantics.

9.4 Context drift

Structured references reduce repetition but can make stale references dangerous. A future resolver should verify existence, version, namespace, and access rights. For mutable context, references may require snapshot IDs or content hashes if deterministic replay is a requirement.

10. Permissions, Safety, and Auditability

Permission representation versus enforcement



Encoding Π makes authority inspectable; it does not grant or enforce authority.

Figure 3. Permission representation versus enforcement. LSTP can represent a sender's requested authority, but effective authority is determined by the consuming system.

10.1 Permission modes

The permissions specification defines an ordered conceptual lattice:

```
RO <= SUGGEST <= PREVIEW <= RW <= EXEC <= COMMIT
```

The ordering expresses increasing operational authority, but a higher mode does not widen scope. `COMMIT` should therefore never be interpreted as “permission to do anything.”

10.2 Scope and forbids

Scope restricts the requested authority to declared resources. Forbids override allowed scope. A consuming runtime should treat ambiguity conservatively and reject requests whose target cannot be resolved safely.

10.3 Limits, confirmation, review, and logging

Cost/time limits and review/confirmation/logging flags are valuable because they turn operational expectations into inspectable data. They still require enforcement mechanisms. A `REQUIRE_CONFIRM=true` field has no protective effect if the runtime ignores it.

10.4 Effective authority

The permission specification's defensible model is:

```
requested permission n local policy n runtime capability
```

This is aligned with least-privilege thinking: the packet expresses no more than a request, while the host remains authoritative.

10.5 Auditability

Auditability requires more than packet logging. A serious implementation should preserve:

- the original carrier representation where policy allows;
- the canonical packet after parsing/compilation;
- validation diagnostics;
- the policy decision and resolved effective authority;
- tool/action requests and results;
- relevant context identifiers;
- version/schema identifiers;
- timestamps and actor identities supplied by the host.

Cryptographic signing, tamper-evident logs, identity assurance, and non-repudiation are separate security capabilities and should not be implied by the word *audit*.

10.6 Evidence and provenance

The `evidence` field can improve traceability if producers populate it accurately. W3C PROV-DM demonstrates that provenance can be modeled around entities, activities, agents, and derivations (Moreau &

Missier, 2013). LSTP's evidence representation is not equivalent to PROV-DM, but a future profile could define mappings for systems that need richer provenance.

10.7 Safety claims

The project should avoid claiming that structured permissions make an AI system safe. Safety outcomes depend on identity, policy, isolation, tool design, verification, monitoring, human oversight, and organizational controls. NIST's AI RMF similarly treats AI risk management as a socio-technical lifecycle concern rather than a single data-field problem (Tabassi, 2023).

11. Integration with SLAI v2.3

Required SLAI v2.3 integration target

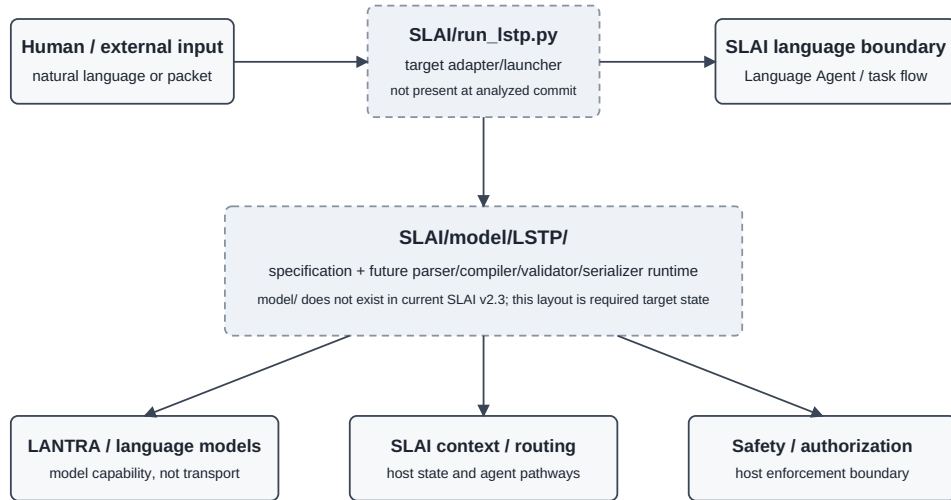


Figure 4. Required SLAI integration target. Dashed elements are required target additions. At the analyzed SLAI commit, neither the root `model/` directory nor `run_lstp.py` exists.

11.1 Repository-grounded current state

The current SLAI branch was inspected at commit `ce4cfe7b1939bc862725e3cb59f488a5b96b1423`. The root contains established SLAI directories such as `src/`, `checkpointing/`, `applications/`, `data/`, `deployment/`, `tests/`, and `training/`. It does **not** contain a root `model/` directory. A direct lookup for `run_lstp.py` also returns no file.

The current `src/agents/language_agent.py` contains SLAI language processing and LANTRA-facing logic, including tokenization, entity/intent processing, context handling, response generation, safety/ethics checks, and configurable LANTRA use. It contains no LSTP adapter or import. Therefore, no current evidence supports describing LSTP as part of SLAI's live language path.

11.2 Required installation architecture

The required target is:

```
SLAI/
├─ run_lstp.py
├─ model/
│   └─ LSTP/
│       ├── README.md
│       ├── LICENSE
│       ├── docs/
│       ├── spec/
│       ├── src/
│       └─ examples/
├─ src/
├─ checkpointing/
├─ applications/
├─ data/
├─ deployment/
└─ ...
```

LSTP should therefore be cloned to `SLAI/model/LSTP/`, while the integration launcher resides at `SLAI/run_lstp.py` rather than inside the LSTP clone.

11.3 Human-to-SLAI flow

A future defensible flow is:

```
human input
-> SLAI/run_lstp.py adapter
-> LSTP parse/compile/validate
-> canonical Octad
-> SLAI language/task boundary
-> SLAI safety/policy/runtime
```

This flow should be optional. Natural language may remain a first-class input path; LSTP should not require users to author formal packets manually.

11.4 Agent-to-agent flow

A future SLAI message envelope could carry a canonical LSTP packet between agents. The receiver should validate version/schema and apply local policy before interpreting requested authority. The packet should complement, not replace, SLAI's task/message metadata unless a migration plan proves equivalence.

11.5 Relationship with LANTRA

LANTRA and LSTP occupy different architectural roles. LANTRA is part of SLAI's language/model infrastructure. LSTP is a proposed semantic representation and transport protocol. A future adapter may use LANTRA to help derive or render semantic packets, but model output must not be treated as automatically schema-valid or authorized. Conversely, an LSTP packet does not improve LANTRA's reasoning merely by existing.

11.6 Safety interactions

LSTP `permissions` should be passed into SLAI policy as requested authority. Existing SLAI safety/ethics checks and future authorization components remain authoritative. If LSTP and SLAI policy disagree, the runtime should choose the more restrictive result or reject the request according to explicit policy.

11.7 Context interactions

LSTP context fields should map to SLAI context only through an adapter that defines ownership and lifecycle. `thread_id` should not be assumed to equal a SLAI task ID, conversation ID, or agent-memory key unless the adapter explicitly establishes the mapping.

11.8 Configuration and launcher behavior

Because `run_lstp.py` is not present, no CLI flags or configuration keys are documented here. When implemented, the launcher should:

1. locate `model/LSTP/` relative to the SLAI root;
 2. refuse incompatible protocol versions rather than guessing;
 3. expose diagnostics clearly;
 4. avoid modifying global import paths in fragile or order-dependent ways;
 5. keep protocol parsing separate from policy enforcement;
 6. be covered by SLAI-side integration tests.
-

12. Implementation State

12.1 Formal artifacts that currently exist

The repository contains substantive versions of:

- `spec/lstp-v0.1.md`;
- `spec/grammar.ebnf`;
- `spec/operator-table.md`;
- `spec/octad_schema.json`;
- `spec/permissions-safety.md`.

`spec/vocabulary.md` is currently minimal and should not be described as a mature vocabulary specification.

12.2 Python module scaffolds

`src/lstp/` contains intended module paths including `packet.py`, `parser.py`, `compiler.py`, `serializer.py`, `validator.py`, `runtime.py`, `diagnostics.py`, `errors.py`, `span.py`, `audit.py`, `canonical.py`, `codes.py`, `cli.py`, and `carrier/format/schema` directories. At the analyzed commit, these files contain no substantive executable implementation.

This means phrases such as “the parser does,” “the serializer guarantees,” or “the CLI supports” would be inaccurate in a present-tense capability description.

12.3 Packaging

`pyproject.toml` is empty. The repository therefore does not currently define a Python package name, dependencies, console script, build backend, supported Python versions, or installation contract. Documentation should not invent `pip install` commands until this metadata exists.

12.4 Examples and tests

An `examples/` scaffold exists, but there is no repository `tests/` directory at the analyzed commit. No current conformance corpus demonstrates parser correctness, schema validity across examples, or carrier round-trip behavior.

12.5 Repository inconsistencies

Several inconsistencies are relevant to implementation quality:

1. the protocol prose references `spec/octad-schema.json`, while the repository file is `spec/octad_schema.json`;
2. the schema `$id` uses `https://example.org/...`, which is appropriate as a placeholder but not a stable project identifier;
3. prior whitepaper prose describes runtime components more concretely than repository code supports;
4. the root README previously used illustrative syntax that does not match the current canonical EBNF packet form;
5. specification and operator-table layers include both canonical Octad syntax and compact notation, but the compiler mapping between them has not yet been executable-tested.

These are correctable maturity issues, not evidence that the protocol concept is invalid.

13. Evaluation and Validation

13.1 Current evidence

No benchmark suite, parser conformance run, round-trip corpus, interoperability trial, or human usability study is present in the analyzed repository. Therefore this whitepaper reports **no performance results**.

13.2 Evaluation framework

A defensible validation program should separate syntactic correctness, semantic preservation, interoperability, usability, security-relevant behavior, and computational cost.

Dimension	Metric	Method	Current result
Parsing	Valid - input parse success	Versioned positive fixture corpus	Not measured
Rejection	Invalid - input rejection precision	Negative/mutation corpus	Not measured
Schema	Octad schema conformance	JSON Schema 2020-12 validator	Not measured
Round trip	Semantic equality after encode/decode	Canonical packet comparator	Not measured
Permissions	Authority preservation	Transformation and policy fixtures	Not measured
Context	Reference resolution accuracy	Synthetic/real conversation graphs	Not measured
Interoperability	Cross-implementation agreement	Independent implementations	Not measured
Inspectability	Human comprehension/error detection	Controlled user study	Not measured
Ambiguity	Explicit unresolved alternatives	Comparative annotation study	Not measured
Overhead	Parse/serialize latency and bytes	Reproducible microbenchmarks	Not measured

13.3 Parse and schema validation

The first evaluation phase should create a versioned fixture corpus. Positive fixtures should cover every grammar production and schema branch. Negative fixtures should include malformed delimiters, invalid atom references, out-of-range confidence, duplicate/conflicting permission fields, invalid identifiers, and unsupported extensions.

13.4 Semantic round-trip integrity

A canonical comparator should define semantic equivalence. Tests should then verify:

```
Octad -> JSON -> Octad  
Octad -> Lattice -> Octad  
Lattice -> Octad -> Lattice -> Octad
```

The final comparison should occur on canonical semantics, not formatting, unless byte-canonical serialization is itself a requirement.

13.5 Ambiguity evaluation

A useful experiment would compare natural-language-only messages with LSTP-mediated representations for a fixed task set. Independent annotators could measure whether intent, scope, evidence, and output requirements are recovered consistently. The experiment should also count cases where formalization introduces incorrect specificity.

13.6 Permission preservation

Property-based tests should attempt transformations across carriers while asserting that mode, scope, forbids, limits, confirmation, review, and logging requirements never become more permissive. Mutation testing can deliberately remove or alter permission fields to ensure validators and policy adapters fail safely.

13.7 Human inspectability

Inspectability should be tested rather than assumed. Participants with different technical backgrounds could identify errors in natural language, JSON Octads, and Lattice packets. Measures might include

task completion time, error-detection rate, subjective workload, and confidence. Results may show that JSON is more inspectable for some users and Lattice for others.

13.8 Interoperability

True protocol interoperability requires at least two independent implementations. Both should consume the same fixtures and produce semantically equivalent canonical packets. A single reference implementation cannot establish interoperability by itself.

13.9 Performance

Only after correctness should the project measure size, parse time, serialization time, memory use, and model-facing tokenization. Token counts should be reported per tokenizer/model because there is no universal mapping from Lattice characters to model tokens.

14. Comparison and Related Work

14.1 JSON and JSON Schema

JSON is a general-purpose data interchange format, while LSTP defines a domain-specific semantic model that can use JSON as a carrier. JSON Schema provides structural validation but does not supply LSTP semantics. LSTP's schema correctly declares Draft 2020-12, whose specification separates core and validation vocabularies (Wright et al., 2022).

14.2 RDF and semantic-web representations

RDF provides an abstract graph model based on subject-predicate-object triples and supports multiple serializations (Cyganiak et al., 2014). LSTP shares the idea that abstract semantics can be separated from serialization, but the Octad is not an RDF graph model. LSTP includes pragmatic, permission, confidence, context, evidence, and output structures that reflect its AI-message focus. Conversely, RDF has mature global identifier and semantic-web ecosystems that LSTP does not currently provide.

14.3 Ontologies

Ontology research emphasizes explicit shared conceptualization and portable vocabularies (Gruber, 1993). LSTP's atoms and relations do not eliminate the ontology problem. Two agents can parse the same packet yet disagree about what a relation type means. Domain vocabularies or mappings remain necessary for strong semantic interoperability.

14.4 Agent communication languages

FIPA ACL and KQML are important precedents for representing communicative acts in agent systems. They separate message-level intent/performative from content and conversation metadata. LSTP's pragmatics and context fields address related concerns, while the Octad additionally elevates confidence, permissions, evidence, and

output structure. No evidence currently establishes that LSTP is superior; the appropriate claim is that it explores a different packet decomposition.

14.5 Domain-specific languages

Lattice is a domain-specific language for semantic messaging. DSL research notes that specialized languages can improve domain expressiveness but impose design, tooling, maintenance, and user-learning costs (Mernik et al., 2005). LSTP must therefore justify its compact syntax through measurable benefits rather than compactness alone.

14.6 Structured prompting and tool/function calling

Modern AI APIs frequently use JSON-like schemas for tool calls and structured output. These mechanisms are typically application- or vendor-specific and focus on invoking functions or constraining output. LSTP's intended scope is broader: communicative pragmatics, context, confidence, evidence, permissions, and output are represented in one canonical packet. That broader scope also increases standardization difficulty.

14.7 RPC and message protocols

Typed RPC and event systems provide mature transport, versioning, serialization, compatibility, and tooling patterns. LSTP should learn from those systems rather than reimplement network transport. A practical architecture can place an LSTP packet inside an existing reliable transport or message bus.

14.8 Provenance models

W3C PROV offers a mature conceptual framework for provenance. LSTP's evidence field is intentionally lighter. If evidence becomes central to high-assurance use, mappings to richer provenance representations may be preferable to expanding LSTP into a second provenance standard.

15. Limitations

15.1 Implementation maturity

The largest limitation is straightforward: the current repository does not contain a working reference implementation. Specification quality can be evaluated, but runtime behavior cannot yet be benchmarked.

15.2 Semantic normalization

The Octad defines where semantic information goes, but not a universal method for deriving the “correct” atoms and relations from language. Different compilers or models may produce different valid packets for the same utterance.

15.3 Residual ambiguity

Formal syntax does not guarantee unambiguous meaning. Relation labels, atom values, domain units, and reference targets can remain ambiguous without shared vocabularies and context.

15.4 Ontology dependence

Interoperability requires more than syntax agreement. Domain vocabularies, stable identifiers, relation semantics, and mapping rules are still needed.

15.5 Schema evolution

The project has not yet established compatibility rules, extension namespaces, migration mechanisms, or version negotiation. These become essential once multiple implementations exist.

15.6 Context drift

References can become stale, inaccessible, or semantically inconsistent as state changes. The protocol does not currently specify snapshot semantics, retention, or conflict resolution.

15.7 Human learning cost

Lattice requires users or developers to learn specialized symbols and field structures. Compactness may reduce readability for newcomers. Human usability has not been studied.

15.8 Security versus permission representation

Permission fields are not security boundaries. They do not provide authentication, authorization, sandboxing, isolation, secret handling, or cryptographic integrity.

15.9 Carrier maturity

Only Lattice grammar and JSON Schema are substantively specified. Additional carriers remain conceptual until mappings and tests exist.

15.10 Benchmarking gap

The project has no empirical evidence for claims about token reduction, model quality, latency, hallucinations, reasoning, or coordination. Such claims should remain hypotheses.

15.11 SLAI integration gap

The required `SLAI/model/LSTP/` layout and `SLAI/run_lstp.py` launcher are target architecture, not current SLAI v2.3 features.

16. Future Work and Roadmap

16.1 Stabilize v0.1

Before expanding the language, the project should resolve file-name inconsistencies, replace placeholder schema identifiers, complete the core vocabulary, and align canonical and compact syntax mappings.

16.2 Implement parser and diagnostics

A tokenizer/parser should be generated or hand-built against the EBNF with explicit source spans and stable diagnostic codes. The parser must reject undefined core syntax rather than infer meaning.

16.3 Implement canonical packet model and compiler

A typed packet model should encode Octad invariants. The compiler should map compact Lattice forms into canonical fields and preserve unresolved constructs as explicit extensions or diagnostics.

16.4 Implement validation and serialization

Validation should combine JSON Schema checks with semantic checks that schemas cannot express. Serializers should define canonicalization before deterministic-hash or signature features are considered.

16.5 Establish packaging and CLI

A populated `pyproject.toml` should define package metadata, dependencies, Python support, and CLI entry points. CLI commands should be documented only after tests verify them.

16.6 Build conformance corpus and CI

Every grammar production, schema branch, diagnostic, and permission transformation should receive fixtures. CI should run parser, schema, round-trip, and mutation tests.

16.7 Integrate with SLAI

Create `SLAI/model/LSTP/` as the clone target and implement `SLAI/run_lstp.py` as an adapter/launcher in the SLAI root. Integration should begin with validation and conversion boundaries rather than direct side-effect execution.

16.8 Evaluate empirically

Run the evaluation program in Section 13. Results should be published with datasets, versions, tokenizers, hardware, and statistical methods sufficient for reproduction.

16.9 Define extension governance

A protocol registry should eventually define core versus extension identifiers, compatibility rules, deprecation, version negotiation, and namespace ownership.

16.10 Additional carriers

Natural-language, binary, multimodal, or specialized carriers should be added only after canonical round-trip behavior is stable. Each new carrier should include an explicit loss model if full semantic preservation is impossible.

17. Discussion

17.1 Compactness versus readability

Lattice can be more compact than verbose JSON, but the compression moves complexity into specialized syntax. Compactness is valuable only if users and systems can still detect errors. The project should therefore treat Lattice and JSON as complementary rather than assuming one is universally preferable.

17.2 Flexibility versus formal rigor

Extensible relation names and attributes make LSTP adaptable, but excessive openness weakens interoperability. Conversely, a fixed global vocabulary would be unrealistic across domains. A core-plus-namespaced-extension model is likely more defensible than unconstrained global terms.

17.3 Expressiveness versus deterministic interpretation

The more expressive a semantic language becomes, the harder deterministic compilation can be. Macros, approximation, contextual references, semantic zoom, and domain operators all increase expressive power while increasing resolver complexity. The protocol should prefer explicit failure over silent inference when deterministic interpretation is unavailable.

17.4 AI efficiency versus human inspectability

A representation optimized for language-model tokenization may differ from one optimized for a human auditor. Because tokenization also varies by model, "AI-efficient" syntax cannot be inferred from character count. LSTP should optimize first for semantic correctness and inspectability, then measure model-facing efficiency.

17.5 Interoperability versus project-specific semantics

The Octad is intentionally project-specific. This provides focus but means external systems need adapters. Interoperability will depend on stable vocabularies, versioning, mappings, and independent implementations. Merely publishing a schema is necessary but insufficient.

17.6 Specification-first development: strength and risk

The current repository's strongest asset is that it has begun stabilizing grammar and semantic contracts before a large runtime exists. This can reduce implementation drift. The corresponding risk is documentation outrunning code, which has already occurred in the older whitepaper. Maintaining explicit maturity labels is therefore not cosmetic; it is part of protocol quality.

17.7 SLAI fit

The current SLAI Language Agent already performs language interpretation, context handling, response generation, and safety-related checks. LSTP should enter that architecture as a transport/representation boundary rather than duplicate language intelligence. LANTRA can remain a model capability, while LSTP can become a schema-controlled interchange form. This separation is architecturally coherent, but it remains to be implemented and tested.

18. Conclusion

LSTP v0.1 is best characterized as a **promising but early specification for structured semantic transport**, not as a completed AI communication runtime. Its current repository provides a meaningful canonical model, formal Lattice grammar, JSON Schema, operator semantics, and a thoughtful permission boundary. The eight-field Octad - pragmatics, atoms, relations, context, confidence, permissions, evidence, and output - creates a concrete basis for making important communication dimensions explicit.

The project's credibility depends on preserving the distinction between what is specified and what is implemented. At the analyzed commit, the Python runtime modules are scaffolds, packaging is undefined, conformance tests are absent, and no benchmark validates claims about ambiguity, token efficiency, reasoning, hallucination, latency, or interoperability. The revised documentation therefore removes those implications and replaces them with measurable evaluation questions.

Integration with SLAI v2.3 is similarly a target rather than a present feature. The required architecture places the LSTP repository at `SLAI/model/LSTP/` and an adapter/launcher at `SLAI/run_lstp.py`; current SLAI contains neither. A future integration should maintain clear boundaries among semantic representation, LANTRA/model capability, SLAI context, and host safety/authorization.

The most valuable next step is not to expand LSTP's claims. It is to implement the smallest conforming parser/compiler/validator/serializer stack, create a rigorous fixture corpus, integrate it through explicit SLAI boundaries, and publish reproducible evaluation results. If those steps demonstrate semantic preservation, usable inspectability, safe permission handling, and cross-implementation agreement, LSTP can progress from an interesting protocol design to a technically validated interoperability mechanism.

References

Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) Data Interchange Format* (RFC 8259). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8259>

Cyганиak, R., Wood, D., & Lanthaler, M. (Eds.). (2014). *RDF 1.1 concepts and abstract syntax*. World Wide Web Consortium. <https://www.w3.org/TR/rdf11-concepts/>

Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2), 199-220. <https://doi.org/10.1006/knac.1993.1008>

Mernik, M., Heering, J., & Sloane, A. M. (2005). When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4), 316-344. <https://doi.org/10.1145/1118890.1118892>

Moreau, L., & Missier, P. (Eds.). (2013). *PROV-DM: The PROV data model*. World Wide Web Consortium. <https://www.w3.org/TR/prov-dm/>

Tabassi, E. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)* (NIST AI 100-1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.AI.100-1>

Wright, A., Andrews, H., Hutton, B., & Dennis, G. (2022). *JSON Schema: A media type for describing JSON documents* (Draft 2020-12). JSON Schema. <https://json-schema.org/draft/2020-12/json-schema-core>

Primary project sources

LSTP project. (2025). *LSTP v0.1 - Protocol & canonical representation specification* [Draft specification]. `spec/lstp-v0.1.md`, analyzed at commit `18d947a45f2db220267f70c7ddb07112fc3181d4`.

LSTP project. (2026). *Lattice grammar* [EBNF]. `spec/grammar.ebnf`, analyzed at commit `18d947a45f2db220267f70c7ddb07112fc3181d4`.

LSTP project. (2026). *LSTP v0.1 operator table* [Normative Phase-1 reference]. `spec/operator-table.md`, analyzed at commit `18d947a45f2db220267f70c7ddb07112fc3181d4`.

LSTP project. (2026). *LSTP v0.1 Octad Packet* [JSON Schema Draft 2020-12]. `spec/octad_schema.json`, analyzed at commit `18d947a45f2db220267f70c7ddb07112fc3181d4`.

LSTP project. (2026). *Permissions and safety specification*. `spec/permissions-safety.md`, analyzed at commit `18d947a45f2db220267f70c7ddb07112fc3181d4`.

SLAI project. (2026). *SLAI v2.3 repository*, branch `SLAI-v.2.3`, analyzed at commit `ce4cfe7b1939bc862725e3cb59f488a5b96b1423`.

Document traceability

Item	State used
LSTP repository	The-0utsider-97/LSTP
LSTP branch	<code>main</code>
LSTP commit	<code>18d947a45f2db220267f70c7ddb07112fc3181d4</code>
SLAI repository	The-0utsider-97/SLAI
SLAI branch	<code>SLAI-v.2.3</code>
SLAI commit	<code>ce4cfe7b1939bc862725e3cb59f488a5b96b1423</code>
Analysis date	21 September 2026
Protocol version discussed	LSTP v0.1
Whitepaper revision	2.0